AOOP Final Project
Advanced Object-Oriented Programming
Halmstad University

David Qvarnström
Sandro Troncoso

May 2023

Abstract

The AOOP project aims to develop a reusable framework for tile games, including sokoban and snake. The architecture is based on the Model-View-Controller (MVC) pattern to ensure ease of use and reusability. The framework offers a consistent approach to application development and accelerates the creation of new games.

List of Contents

1 Introduction.

The specification of this project is to create a program that utilizes a versatile framework capable of implementing more than one type of application by reusing existing code. By utilizing the MVC pattern and making the model abstract, an application can be linked to the framework. The necessary methods will then be implemented to create the application. By separating the application and framework, the program is more balanced, and it is easier to find potential issues with the code.

This project is part of Advanced Object-Oriented Programming (DT4014, 7.5 credits), a course given at Halmstad University to computer engineering students.
The report starts by covering the program's design and implementation. It then discusses the testing process and how the tests helped improve or resolve issues in the initial approach. The middle of the report contains a Discussion section where interesting topics regarding the program and the logic of saving/loading are discussed, followed by the Results section which highlights findings and realizations regarding the design and ideas.
The Version Control section contains the group's experiences using GitHub for version control of the program, followed by a Reference List section.

2 Design.

2.1 Tile Games

When creating a typical tile game, it will have some standard functions, e.g., a visual representation of the game, i.e., a function that controls the player's movement and something that keeps track of the state of the game. This framework aims to provide reusable components and modules that tile games have in common. The primary purpose is to speed up a new game's development and enforce a more consistent way of creating games with the framework. Ease of use and reusability was the main goal when developing the project.

2.2 MVC pattern

The pattern is often used as the core of game development[1, 4]. The design pattern splits up the game into three parts.

The Model is responsible for the logic of the game. It handles the different game states, Game mechanics and is responsible for data handling and saving/loading.

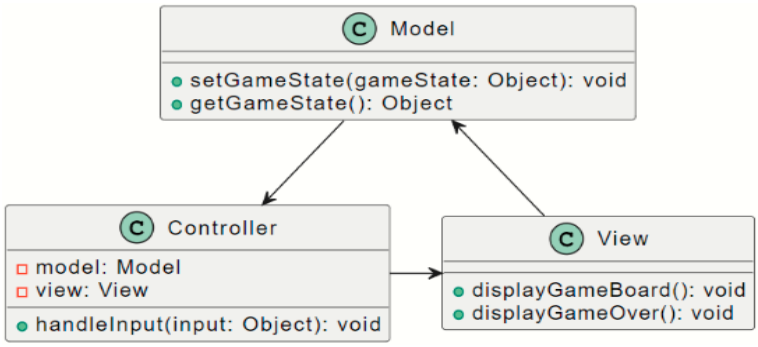

Figure 1: MVC pattern

View is responsible for presenting the data from the Model to the user. It's responsible for the graphical representation of the game. The controller acts as an intermediary between the Model and the View and is responsible for the input from the user. [1]

2.3 Modifying the MVC

The project has the intention of reusable code, and the framework/application design fits well into our intentions. The MVC(Model-View-Controller) pattern is the project's core which is then modified to work as the framework part.[1] Making the Model abstract allows an application programmer to reuse code often used when creating tile games. By doing this, the programmer does not have to worry about how the tiles are made and stored.

Providing a easy way to create and maintain tiles by using JLables as tiles make it easy to create a board of almost any size and add tiles with images or text. The tiles are maintained using the prototype pattern. [2]

The board is designed to be flexible thanks to the use of JLabels and a GridBagLayout. This allows it to take many different shapes without any issues. [3]

To separate the application from the framework and provide a set of generic functionalities that all tile games share, such as input handling, observer management, tile and board creation and provide a framework that can be used to quickly create a simple game and test different inputs or outputs. Using the MVC pattern as a core, adding functions for adding and removing in/output, and creating a model class that tile games can reuse.

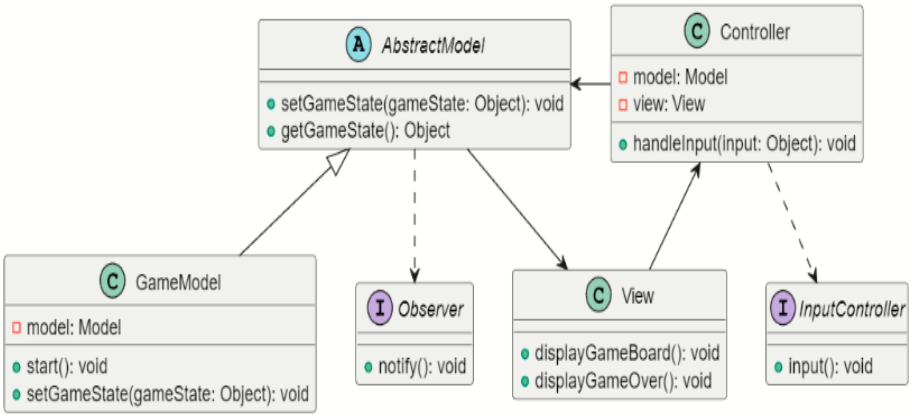

Figure 2: Modified MVC pattern

2.4 Observer pattern for input

This pattern is used when a class creates data that other parts of the program may be interested in. The class that produces information is called the subject; in this case, the AbstractModel is the subject. To easily create and attach new observers that are decoupled from each other.

The pattern uses an interface to enable polymorphism, and classes that want to be notified when updates occur to the board in the AbstractModel have to implement the Observer interface.[3, 4]

```java
/**
 * Observer interface for the game. This interface is used to notify the
 * observer of changes in the game. The passed parameters are the game board and
 * the state of the game.
 */
public interface GameObserver {

    public void notify(int[][] board, GameStateAndDirection stateUpdate);
}
```

Figure 3: Observer Interface

The Subject, AbstractModel, has a list of all the observers that want to be notified when updates occur. The subject also has methods for adding and removing observers.

The way a specific Observer uses the information is implemented in the class. In this PrintOut class, implementing the notify method produces a matrix representing the updated state of the game in the console. But it can also be used to play sound or saving the game states and input directions. This class or any class that implements the interface can easily be added or removed from the list in the subject class, making the PrintOut class decoupled from the rest of the program.

```java
/**
 * PrintOut is an observer that prints the game board and the state of the game
 * to the console.
 */
public class PrintOut implements GameObserver {

    @Override
    public void notify(int[][] board, GameStateAndDirection direction) {
        for (int i = 0; i < board.length; i++) {
            System.out.println();
            for (int j = 0; j < board[0].length; j++) {
                System.out.print(board[i][j]);
            }
        }
        System.out.println();
        System.out.println(direction);
    }
}
```

Figure 4: PrintOut Observer

2.5 Strategy pattern for input

The Strategy pattern[5, 3] is often used when a class wants to benefit from different variants of an algorithm, e.g., controlling a character in a game. The context in this design is the GameController class that wants to benefit from other algorithms.
The GameController class should be able to receive inputs from several types of input from the user, such as keyboard input, mouse input, or others. Each input is encapsulated within a separate strategy class, concrete strategy, that implements the InputController interface. This design choice allows for easy extensibility by adding new inputs or sharing inputs with other games using the same framework.
To use the strategy pattern, the class must implement an interface and use Enums in the GameStateAndDirectory to pass new instructions to the Model.

```
/**
 * InputController is an interface for the input to GameController. The input
 * controller is responsible for getting input from the user and passing it to the model.
 * The input controller is using the method pattern to implement the input.
 */
public interface InputController {
    public GameStateAndDirection input();
}
```

Figure 5: Strategy Interface

The framework has been designed to provide ease of use and includes a set of Enums within the GameStateAndDirectory class. The application programmer must utilize these Enums to provide new input to the Model. These Enums, including UP, DOWN, LEFT, RIGHT, GAME PAUSE, and MUTE, make the framework more user-friendly. For implementing keyboard input, the programmer should use the interface to implement the input method. After implementation, the programmer must pass an Enum from the GameStateAndDirection class to the controller in the framework. The controller will then pass the Enum to the model, which allows for efficient and streamlined keyboard input configuration.

2.6 Prototype pattern for tiles

To handle the creation of tiles (represented by JLabels), the program uses the prototype pattern[4]. A TileRegistry class is used as the registry of prototype tiles, which are pre-created instances of each tile type used by the game. When a new tile is needed, the TileRegistry clones the prototype tile that is needed and puts it on the board. Although this design pattern is not essential for making the framework usable, it has been implemented due to its interesting structure.

2.7 Memento Patter to save and load

The Memento pattern[5, 2] can be explained using the analogy of post-it notes as bookmarks in a book. Imagine you are reading a book and want to remember where you left off. You can use a post-it note to mark the page. Later, you can return to the book, remove the post-it note, and continue reading from where you left off.

In the same way, the Memento pattern allows you to save the state of an object and restore it later. This is useful for games and other applications where the user may want to save their progress and continue later. The Memento pattern involves creating a separate class to store the state of the object, which can then be saved to a file or database. When the user wants to restore the state, the saved state can be loaded into the object.

The ‘GameSaved‘ class in the framework is an example of the Memento pattern. This class stores the game state, including the board matrix, score, and level. The class implements ‘Serializable‘, which allows it to be serialized and saved to a file. When the game is saved, an instance of ‘GameSaved‘ is created and populated with the necessary information. The instance is then serialized and saved to a file. To restore the game state, the serialized file is loaded, and the ‘GameSaved‘ instance is deserialized. The board, score, and level are retrieved from the deserialized object and used to restore the game state.

3 UML Diagram.

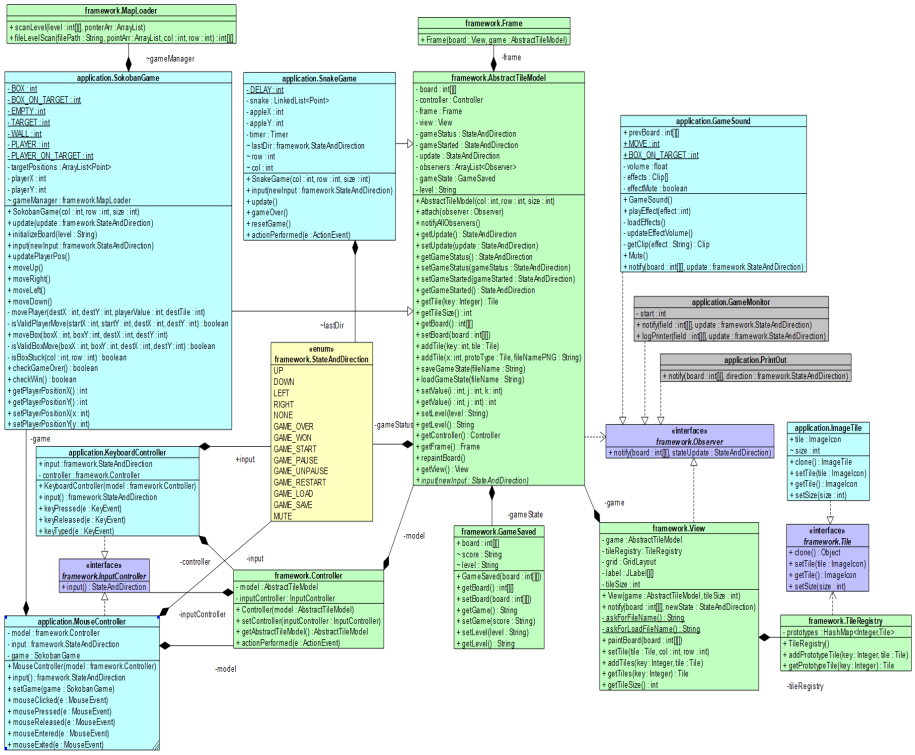

Figure 6: UML Diagram

4 Testing.

During the development of this project, several tests have been carried out to ensure that the methods and approaches have been acting as intended. When the project was at the end stages, JUint was used to test the program's functionality.

One of the tests was to stress test and check if the Sokoban game would experience any unexpected problems while making random movements across a map, such as clipping through the walls or not getting game-overs. The different directions were implemented to be executed by a random value between zero and one by using the Math.random method. The method checkgameoverandclip checks if the game is over or if the player's position is out of the map's bounds.

```
/*
 * Make the player move randomly and count the number of times the game was won and lost.
 * */
@Test
public void testHighNumberORandomMovement() {

    SokobanGame soko =  new SokobanGame(13, 13, 50);

    int lost = 0;
    int won = 0;
    int moves = 0;

    for (int i = 0; i < 5000000; i++)
    {
        double random = Math.random();

        if(random >= 0 && random <= 0.249) {
                soko.input(StateAndDirection.UP);
                if(soko.checkWin())
                    won++;
                if(checkgamoverandclip(soko));
                    lost++;
        }
        if(random >= 0.250 && random <= 0.499) {
                soko.input(StateAndDirection.DOWN);
                if(soko.checkWin())
                    won++;
                if(checkgamoverandclip(soko));
                    lost++;
        }
        if(random >= 0.500 && random <= 0.749) {
                soko.input(StateAndDirection.LEFT);
                if(soko.checkWin())
                    won++;
                if(checkgamoverandclip(soko));
                    lost++;
        }
        if(random >= 0.750 && random < 1) {
                soko.input(StateAndDirection.RIGHT);
                if(soko.checkWin())
                    won++;
                if(checkgamoverandclip(soko));
                    lost++;
        }
        moves++;
    }
    System.out.println("Game won: " + won + " times");
    System.out.println("Game lost: " + lost + " times");
    System.out.println("Random moves made: " + moves + " times");


}
```

Figure 7: Test Random Movement

When the player moves in different directions, the method checks if the game has been won or lost. Depending on the result, a local variable would increment. After the test, the result gets printed in the console window.

Another test was to test if the game's initial state were correct and if the player could move boxes and not clip through the boxes or walls. When the game results in a game over, the test checks if the game has reset by using the assertValue method on the X and Y coordinates of the character where the start position is expected.

```
@Test
public void testGameInitAndCrateStuckGameOverThenReset() {


    SokobanGame soko =  new SokobanGame(13, 13, 50);

    /*
     * Check start position
     * */
    assertEquals(soko.getPlayerPositionX(),6);
    assertEquals(soko.getPlayerPositionY(),6);

    /*
     * Check if the player will stand in the tile above the crate
     * when the crate have been pushed against the wall of map 1.
     * */
    for (int i = 0; i < 100; i++) {
        soko.input(StateAndDirection.DOWN);
    }
    assertEquals(soko.getPlayerPositionX(),6);
    assertEquals(soko.getPlayerPositionY(),10);

    /*
     * Move player one step to the left and check position
     * */
    soko.input(StateAndDirection.LEFT);
    assertEquals(soko.getPlayerPositionX(),5);
    assertEquals(soko.getPlayerPositionY(),10);

    /*
     * Move player down to make the player stand to the left tile
     * from the crate and check that the player won't clip through the wall
     * */
    for (int i = 0; i < 100; i++) {
        soko.input(StateAndDirection.DOWN);
    }
    assertEquals(soko.getPlayerPositionX(),5);
    assertEquals(soko.getPlayerPositionY(),11);

    /*
     * Push the crate against the wall and check if the game is over.
     * */
    for (int i = 0; i < 5; i++) {
        soko.input(StateAndDirection.RIGHT);
        if(soko.getPlayerPositionX() == 10)
        assertTrue(soko.checkGameOver());
    }

    /*
     * Check if the map have been reset and the player is
     * in the start position of map 1.
     */
    assertEquals(soko.getPlayerPositionX(),6);
    assertEquals(soko.getPlayerPositionY(),6);

}
```

Figure 8: Game over and position JUnit test

For every action the player makes, the test checks the player's coordinates on the map to assert that the player is in the correct position based on the rules and map layout the test has been running on.

5 Discussion.

When using the memento pattern, there are some considerations to keep in mind when implementing it.

The "save" function sets the name of the saved game as the title of the currently running game. However, this can easily be bypassed by changing the name[5]. To ensure that the correct game is loaded, we also save the name of the currently running class into the "memento". When the game is loaded, it checks if the saved class name matches the name of the currently running program.

```
String className = this.getClass().getSimpleName();

gameState.setGame(className);
```

Figure 9: Save the Class name into memento

```
if(temp.getName().equals(this.getClass().getSimpleName()) == false){
    System.out.println(x:"Gamestate not compatible, wrong game");
    return;
}
```

Figure 10: Checks the game of the loaded memento

When using the memento pattern to save the state of the snake game, a problem arises when the board can be of any size, unlike in Sokoban where all maps are the same. To resolve this issue, the load method has to check that the memento has a board of the same size as currently used. Alternatively, a better way could be to construct a new board of the correct size.

```
if(temp.getBoard().length != board.length || temp.getBoard()[0].length != board[0].length){
    System.out.println(x:"Gamestate not compatible, wrong board size");
    return;
}

for(int i = 0; i < temp.getBoard().length; i++){
    for(int j = 0; j < temp.getBoard()[0].length; j++){
        board[i][j] = temp.getBoard()[i][j];
    }
}
```

Figure 11: controlling the size of the loaded game board

6 Results.

The goal of the AOOP project was to create a reusable framework that could handle Sokoban and Snake. By utilizing inheritance, encapsulation, abstraction and polymorphism, this framework can be used to implement other tile-based games without requiring knowledge of the core code structure.
The framework was designed with a modified MVC pattern and with an abstract model that defines the common methods and properties that tile games use. This approach allows for greater flexibility and modularity, making it easy to add new games to the framework.
To solve the specifications that came with the project we utilized different design patterns like observer-pattern, method-pattern, memento-pattern, and prototype-pattern. Multiple games can be run at the same time without interference which is a good feature. Overall, this framework enables quick creation of flexible boards for tile games, making it ideal for testing game ideas.

7 Version Control.

Using a code hosting platform for version control and collaboration has been a significant help throughout this project. It enabled the team to check for changes in the program made by collaborators in the group before pulling the latest version, and this includes not only the code but also files such as images and sound.
Not only is it possible to check the latest version, but the fact that it was possible to go back to a previous version and check the changes could sometimes resolve error messages which could be overlooked after closing the IDE, making the programmer unable to go back to an earlier state.
It also enabled the team to create different branches which could be used as an idea storage or testing area where code snippets or methods of interest could be saved to be implemented at another time with a different approach that suited the program better. [6]

8 Reference List.

[1] Veronica Gaspes. *Lab Material from the Programming (DT2012) course at Halmstad University 2021*. 2021.

[2] GeeksForGeeks. *GeeksForGeeks - Memento Pattern*. 2023. URL: `https://www.geeksforgeeks.org/memento-design-pattern/` (visited on 05/2023).

[3] Cay Horstmann. *TB Object-Oriented Design Patterns*. 2006, pp. 1–475. ISBN: 0471744875.

[4] Wojciech; Nesma Rezk Mostowski. *Lectures from the Advanced Object-Oriented Programming (DT4014) course at Halmstad University 2023*. 2023.

[5] Wiki. *Memento_pattern*. 2023. URL: `https://en.wikipedia.org/wiki/Memento_pattern` (visited on 05/2023).

[6] Wikipedia. *Wikipedia - GitHub*. URL: `https://en.wikipedia.org/wiki/GitHub` (visited on 05/2023).